Experiment 3

Student Name: Shubham Sundram
UID: 22BCS17273
Branch: CSE
Section/Group: 627-A
Semester: 6th
Date of Performance: 31/01/25
Subject Name: Advanced Programming Lab-2
Subject Code: 22CSP-351

1. **Aim:**

a) You are given the heads of two sorted linked lists list1 and list2. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.

b) Given the head of a sorted linked list, delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list. Return the linked list sorted as well.

c) Given the head of a singly linked list, reverse the list, and return the reversed list.

2. **Objective:**

a) Merge two sorted linked lists list1 and list2 into a single sorted list by splicing together their nodes. Return the head of the merged linked list.

b) Remove all nodes containing duplicate numbers from a sorted linked list, ensuring only distinct numbers remain. Return the modified sorted linked list.

c) Reverse the pointers of the given singly linked list so that the last node becomes the head. Achieve this in O(n) time and O(1) space by iterating through the list and updating links.

3. **Algorithm:**

a) Algorithm for Merge two linked list: -

1) Create a dummy node as the starting point and use a current pointer to track the last node of the merged list.
2) Compare the values of list1 and list2, append the smaller value to the new list, and move the corresponding list pointer forward.
3) Repeat the process until one of the lists is exhausted.
4) Attach the remaining nodes of either list1 or list2 directly to the merged list.
5) Return dummy.next as the final merged linked list.

b) Algorithm for Remove duplicates from linked lists: -

1) Initialize a dummy node to handle edge cases.
2) Use a pointer prev to point to the dummy node and current to traverse the list.
3) Check for duplicate values by comparing the current node's value with the next node.
4) If duplicates are found, skip all nodes with that value.
5) If no duplicates are found, update the prev pointer to point to the current node.
6) Return the next node of the dummy (which represents the start of the updated list).

c) Algorithm for reverse a linked list: -

1) Initialize a pointer node to nullptr, which will store the reversed list.
2) Iterate through the linked list while head is not nullptr.
3) Store the next node in a temporary pointer temp to keep track of the remaining list.
4) Reverse the current node's pointer by making head->next point to node, then update node to head and move head to temp.
5) Once the loop ends, return node as the new head of the reversed list.

4. Implementation/Code:

a) Code for merge two sorted list:-

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode* dummy = new ListNode(0);
        ListNode* cur = dummy;

        while (list1 && list2) {
            if (list1->val > list2->val) {
                cur->next = list2;
                list2 = list2->next;
            } else {
                cur->next = list1;
                list1 = list1->next;
            }
            cur = cur->next;
        }
        cur->next = list1 ? list1 : list2;
        ListNode* head = dummy->next;
        delete dummy;
        return head;
    }
};
```

b) Code for Jump Game II problem:-

```
class Solution {
public:
    ListNode* deleteDuplicates(ListNode* head) {
        ListNode dummy(0, head);
        ListNode* prev = &dummy;
        while (head) {
            if (head->next && head->val == head->next->val) {
                while (head->next && head->val == head->next->val) {
```

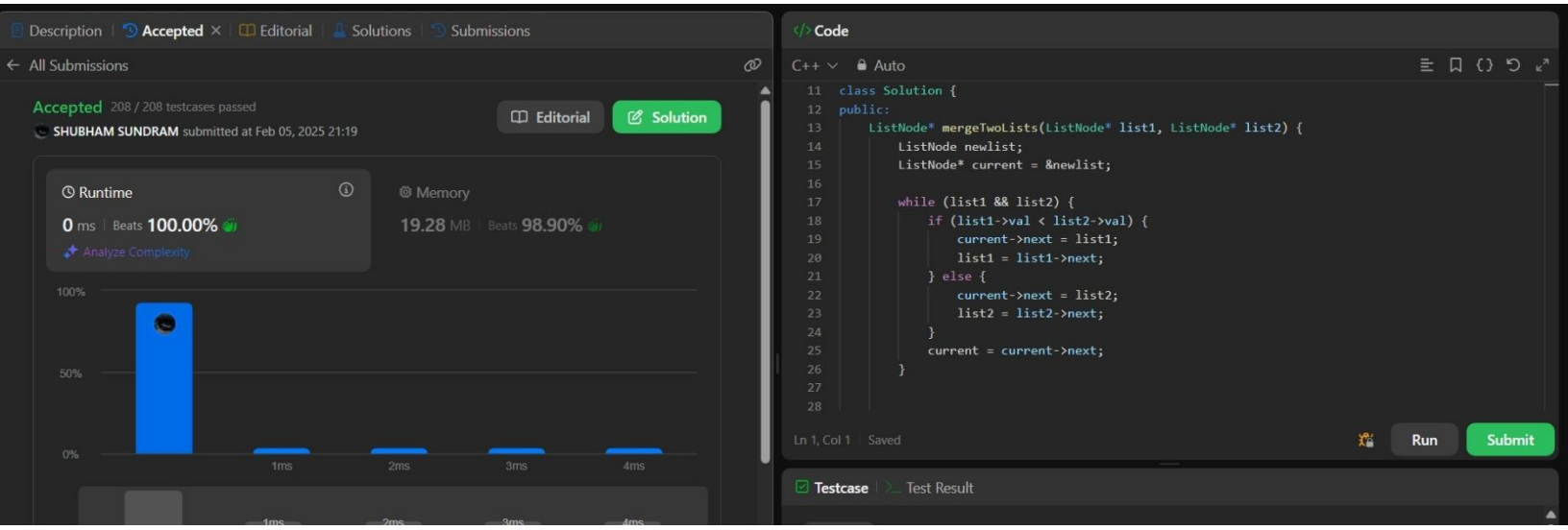
```
                head = head->next;
            }
            prev->next = head->next;
        } else {
            prev = prev->next;
        }
        head = head->next;
    }
    return dummy.next;
}
};
```

c) Code for Reverse a Linked List: -

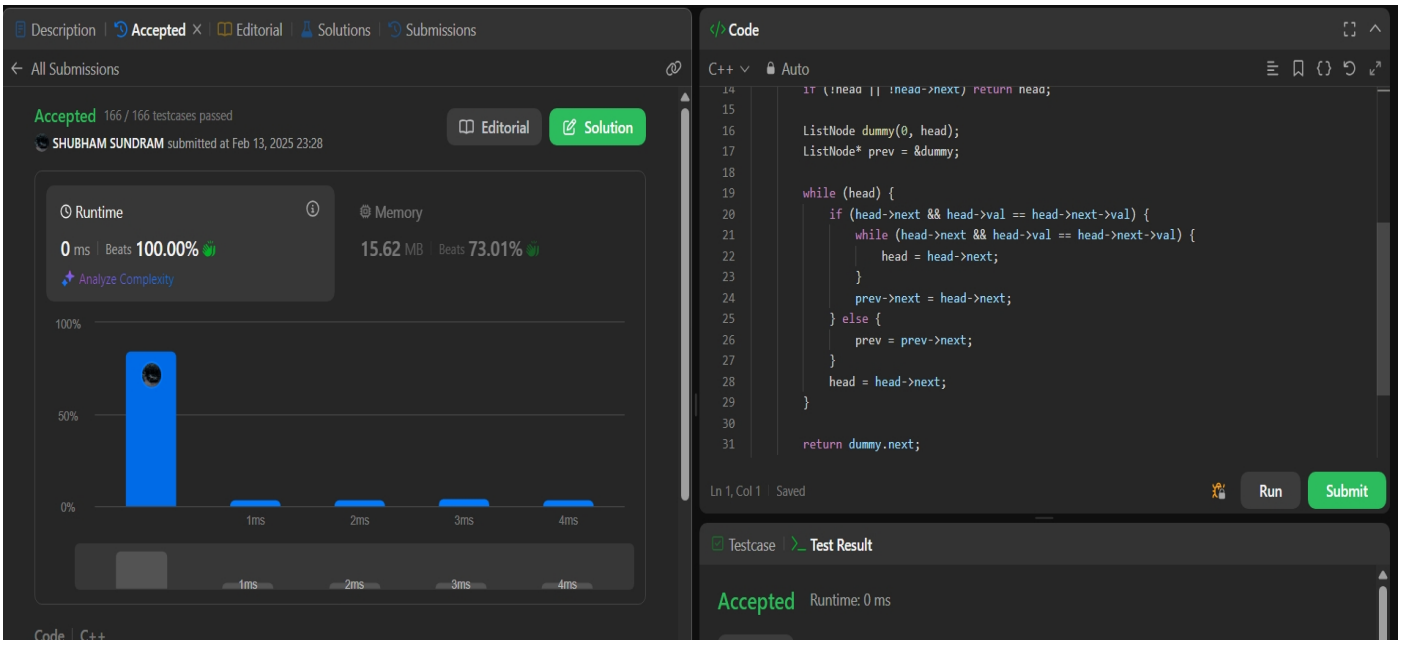
```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* node = nullptr;
        while (head != nullptr) {
            ListNode* temp = head->next;
            head->next = node;
            node = head;
            head = temp;
        }
        return node;
    }
};
```

5. Output:

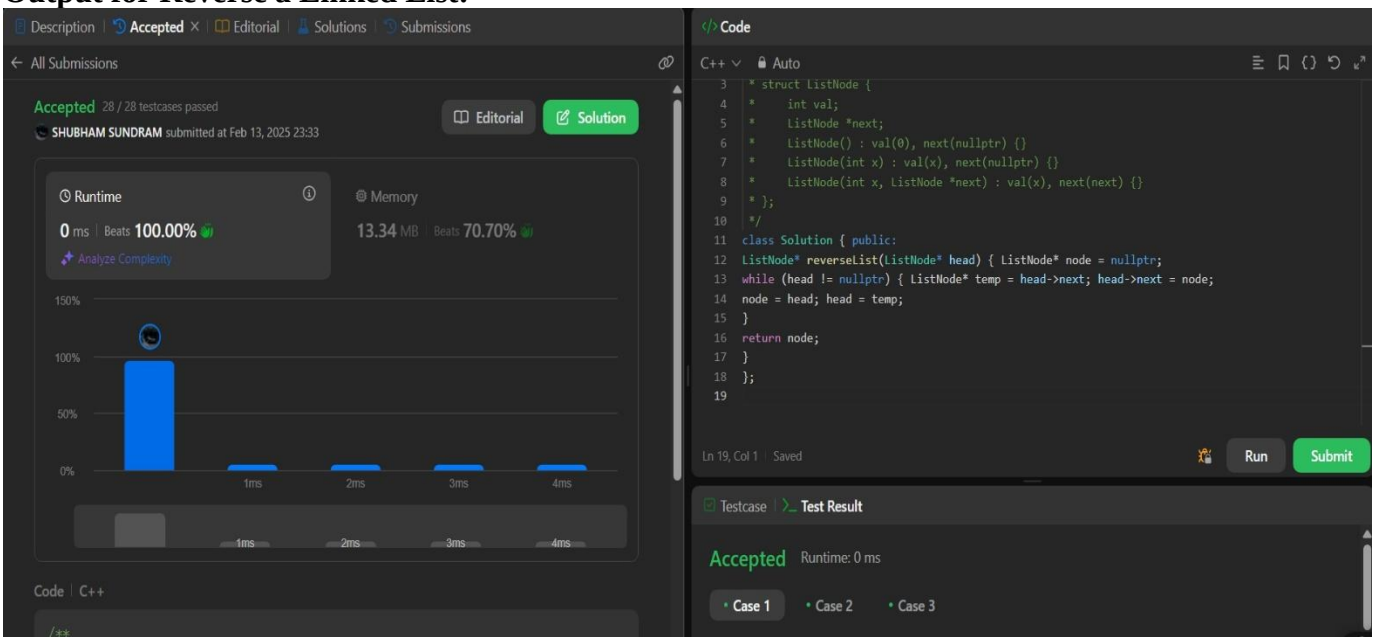
a) Output for Merge two sorted: -



b) Output for Remove Duplicates: -



c) Output for Reverse a Linked List: -



6. Learning Outcomes:

- Understand and implement the merging of two sorted linked lists efficiently while maintaining the sorted order.
- Develop problem-solving skills in linked list manipulation, including node splicing and pointer management.
- Gain proficiency in detecting and removing duplicate nodes from a sorted linked list while preserving the order.
- Enhance knowledge of linked list traversal techniques and in-place modifications.